

Since the application now has PWA support, it will continue to render even if we go offline, since the assets have been cached by the service worker running behind the scenes.

Up to this point we have only learned about how to cache the static assets of the Web application. We haven't integrated any REST API with the application, and made the necessary changes to cache the API response.

Let's integrate a REST API with the Angular application and modify *ngsw-config.json* to cache the API response.

To create any kind of connection to the outside world (REST APIs) we use a construct defined in the Angular domain, called a service.

To create a service in the app, we run the following command:

```
ng g service DataService
```

This will generate a service file named *data-service.service.ts*, and a test file, which contains the code needed to interact with the REST API. For our example application, we used a service to return Lorem Ipsum text. Contents of the *data-service.service.ts* file are shown in the code snippet below:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root'
})
export class DataServiceService {

  constructor(private http:HttpClient)
  { }

  getRandomData(){
    return this.http.get('http://www.randomtext.me/api/');
  }
}
```

We also update the *ngsw-config.json*

to add the instruction to cache the API response. The *pdated ngsw-config.json* is given below:

```
{
  "$schema": "./node_modules/@angular/service-worker/config/schema.json",
  "index": "/index.html",
  "dataGroups": [
    {
      "name": "api-performance",
      "urls": ["/assets/i18n/**", "/api/**"],
      "cacheConfig": {
        "strategy": "performance",
        "maxSize": 100,
        "maxAge": "3d"
      }
    },
    {
      "name": "api-freshness",
      "urls": ["http://www.randomtext.me/api/"],
      "cacheConfig": {
        "strategy": "freshness",
        "maxSize": 100,
        "maxAge": "3d",
        "timeout": "10s"
      }
    }
  ],
  "assetGroups": [
    {
      "name": "app",
      "installMode": "prefetch",
      "resources": {
        "files": [
          "/favicon.ico",
          "/index.html",
          "/manifest.webmanifest",
          "/*.css",
          "/*.js"
        ]
      }
    },
    {
      "name": "assets",
      "installMode": "lazy",
      "updateMode": "prefetch",
      "resources": {
        "files": [
          "/assets/**",
          "/*.(eot|svg|cur|jpg|png|webp|gif|otf|ttf|woff|woff2|ani)"
        ]
      }
    }
  ]
}
```

Now we build the application again using *ng build—prod* and run it using *http-server -p 8080 -c-1 dist/angular-pwa-seed*.

Thus we first see the output without going offline.

Next, we toggle the offline radio button and see if the application still works.

And that is it. We have successfully converted an Angular application to a PWA. Web application. All the code that we have shown here can be found on the GitHub repo *ang-angular-pwa-seed*.

PWAs are the future — where you write your native-like applications using JavaScript, without having to go through difficult build steps and frustrating app stores. With nearly full access to a device's native functionality, PWAs are bound to take over from native applications and become the No.1 way of creating incredible experiences on all devices. **END** 🐧

🔗 Reference

<https://ionicframework.com/docs/v3/developer-resources/progressive-web-apps/>

👤 By: Abhinav Nath Gupta

The author is a software development engineer at Cleo Software India Pvt Ltd, Bengaluru, and is interested in cryptography, data security, cryptocurrency, cloud computing and Web application development.